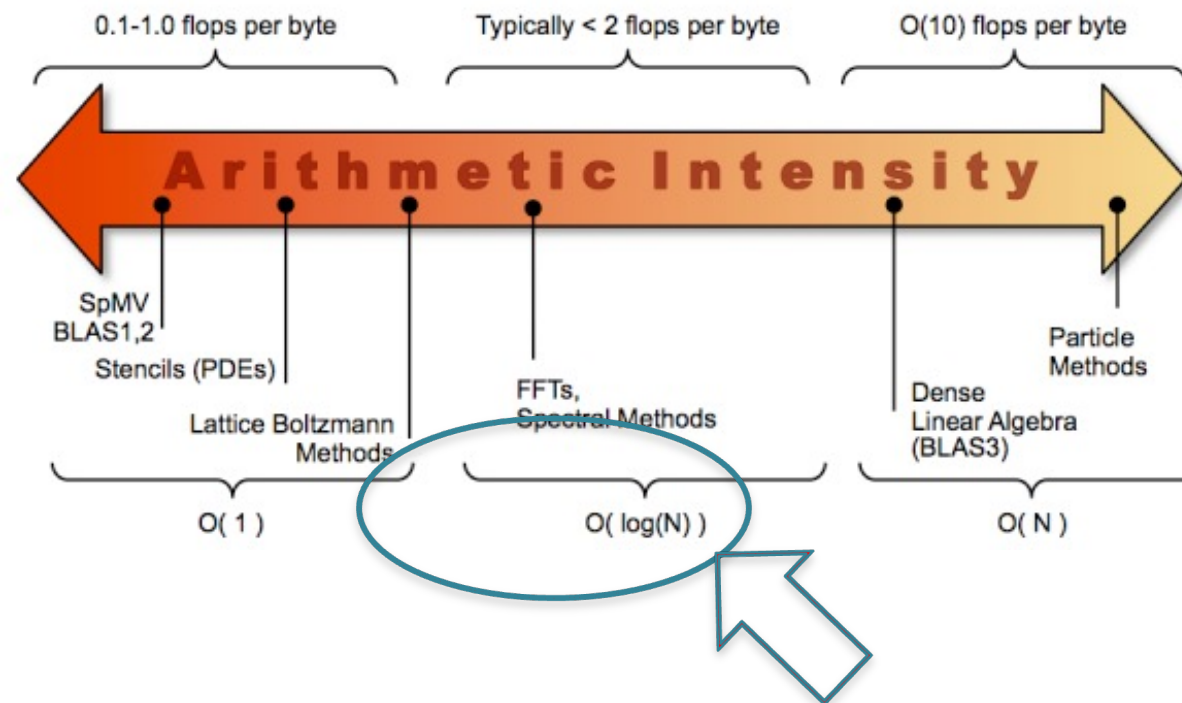


Parallel Sorting

Parallel Algorithms

João Luís Sobral (jls@di ...)

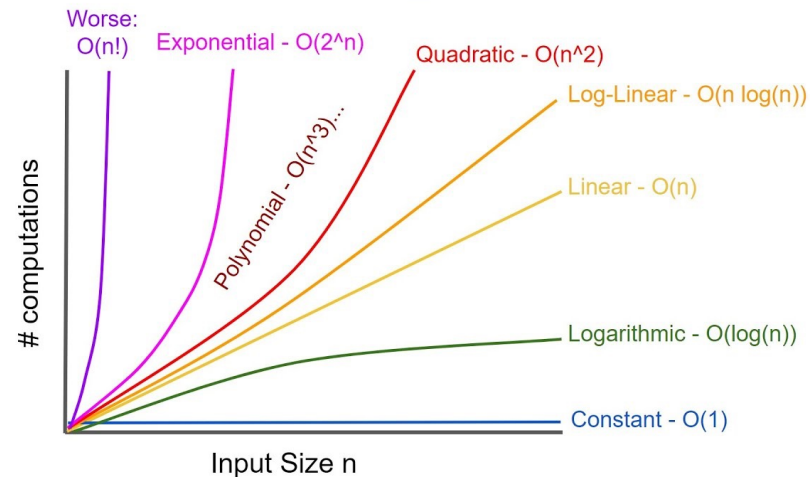
March/2026



Parallel Algorithms - recap

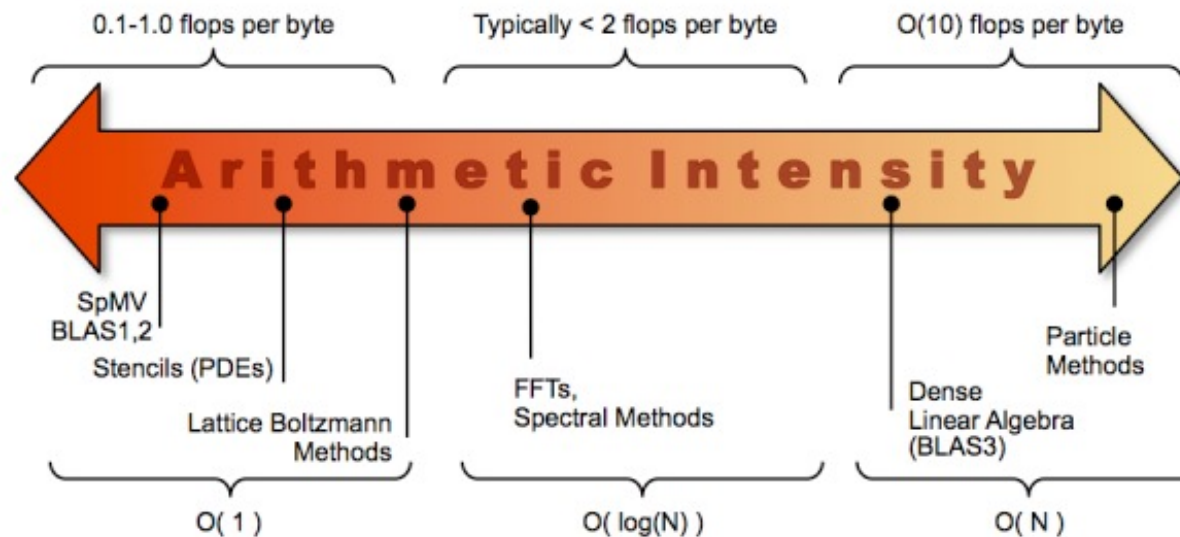
- **Traditional algorithm analysis: Big O notation**
 - Analysis of the number of operations per datum (n)
 - **Particle Methods (naïve!): $O(n^2)$**
 - Matrix multiplication: $O(n^3)$ $O(n^{\frac{3}{2}})$ (n =number of elements of C)
 - **Sorting (n keys)**
 - Bad sorting (brute force): $O(n^2)$
 - Better algorithm: $n \log_2(n)$
 - Best: $([k]n)$

← Today's lecture



Parallel Algorithms

- **Arithmetic Intensity** (operations per ~~byte loaded~~^{data moves})



- **Data storage**

- Regular : vector, matrix
- Irregular (pointer based): tree, graphs



Data accesses

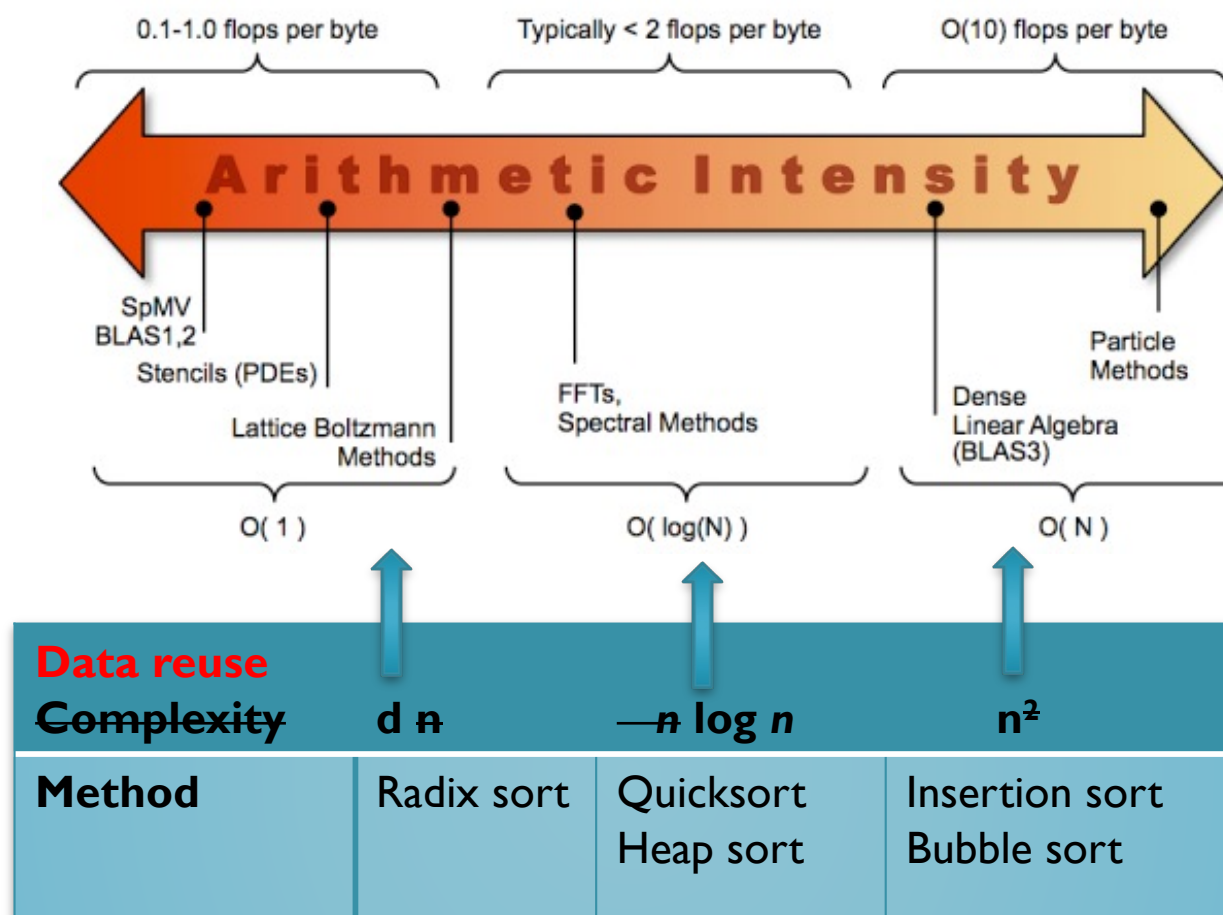
- Regular / irregular

Parallel Sorting

- [Sequential] sorting algorithms

Method	Complexity (average)	Description
Insertion sort	n^2	Insert elements into a sorted list
Bubble sort	n^2	Compare (and swap) successive elements
Quicksort	$n \log n$	Recursively sort elements less/greater than a given pivot
Merge sort	$n \log n$	Successively merge sorted sub-lists starting from lists with one element
Heap sort	$n \log n$	Insert elements into a binary heap
Radix sort	$n d$	Sort elements digit by digit (d) (two variants: MSD and LSD)

Parallel Sorting



Parallel Sorting

- Locality: Radix MSD vs Radix LSD
 - Both are based on key-indexed sorting
 - Example for 3-digit Strings (figure: 1st digit step on MSD)

1. Count frequencies of each letter using key as index
2. Compute frequency cumulates
3. Access cumulates using key as index to find record positions.
4. **Copy** back into original array

```
int N = a.length;
int[] count = new int[R];

count frequencies → for (int i = 0; i < N; i++)
                    count[a[i]+1]++;

compute cumulates → for (int k = 1; k < 256; k++)
                    count[k] += count[k-1];

move records → for (int i = 0; i < N; i++)
                temp[count[a[i]++]] = a[i]

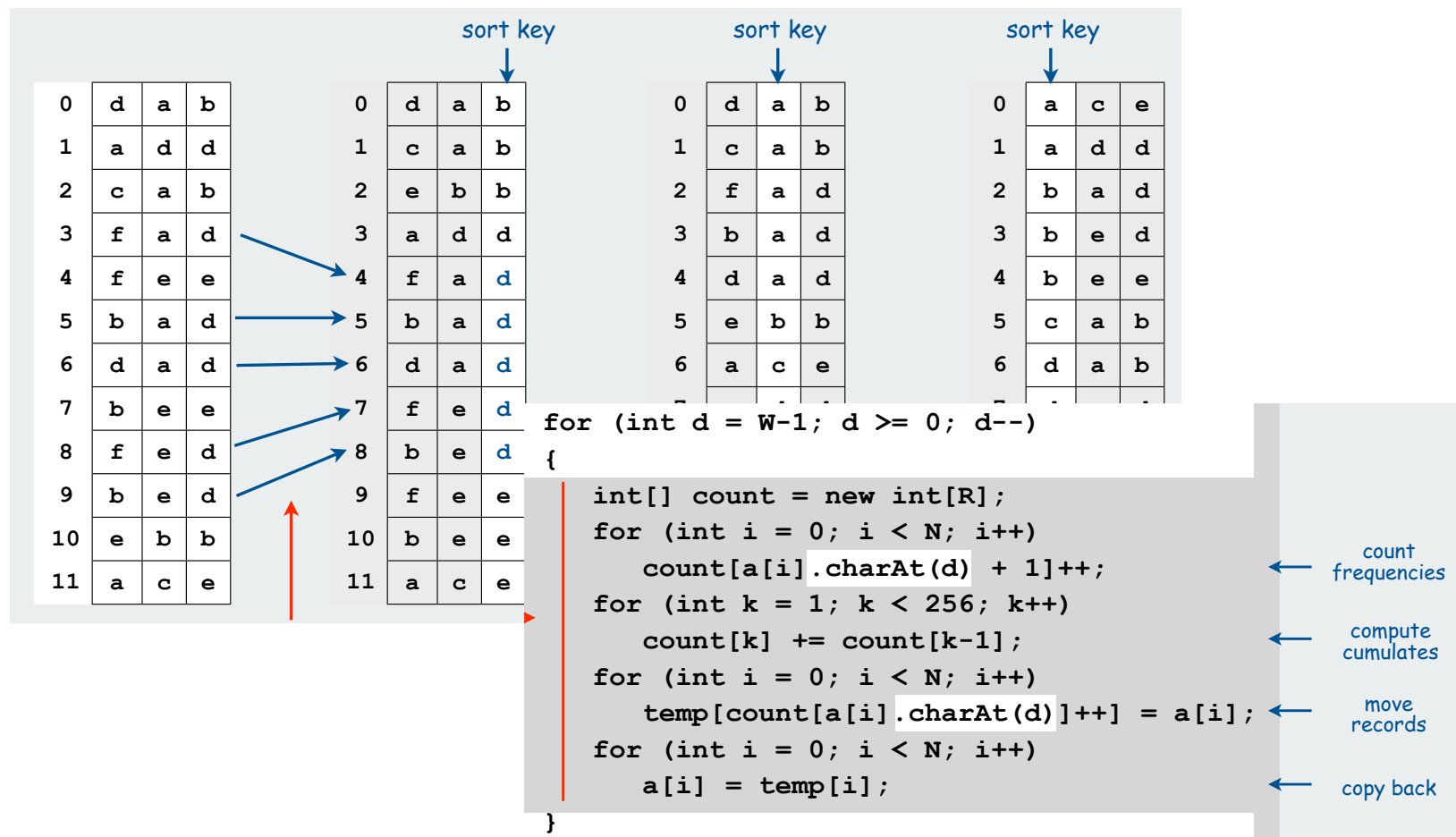
copy back → for (int i = 0; i < N; i++)
             a[i] = temp[i];
```

a[]			temp[]	
0	a		0	a
1	a		1	a
2	b		2	b
3	b		3	b
4	b		4	b
5	c		5	c
6	d		6	d
7	d		7	d
8	e		8	e
9	f		9	f
10	f		10	f
11	f		11	f

count[]	
a	2
b	5
c	6
d	8
e	9
f	12

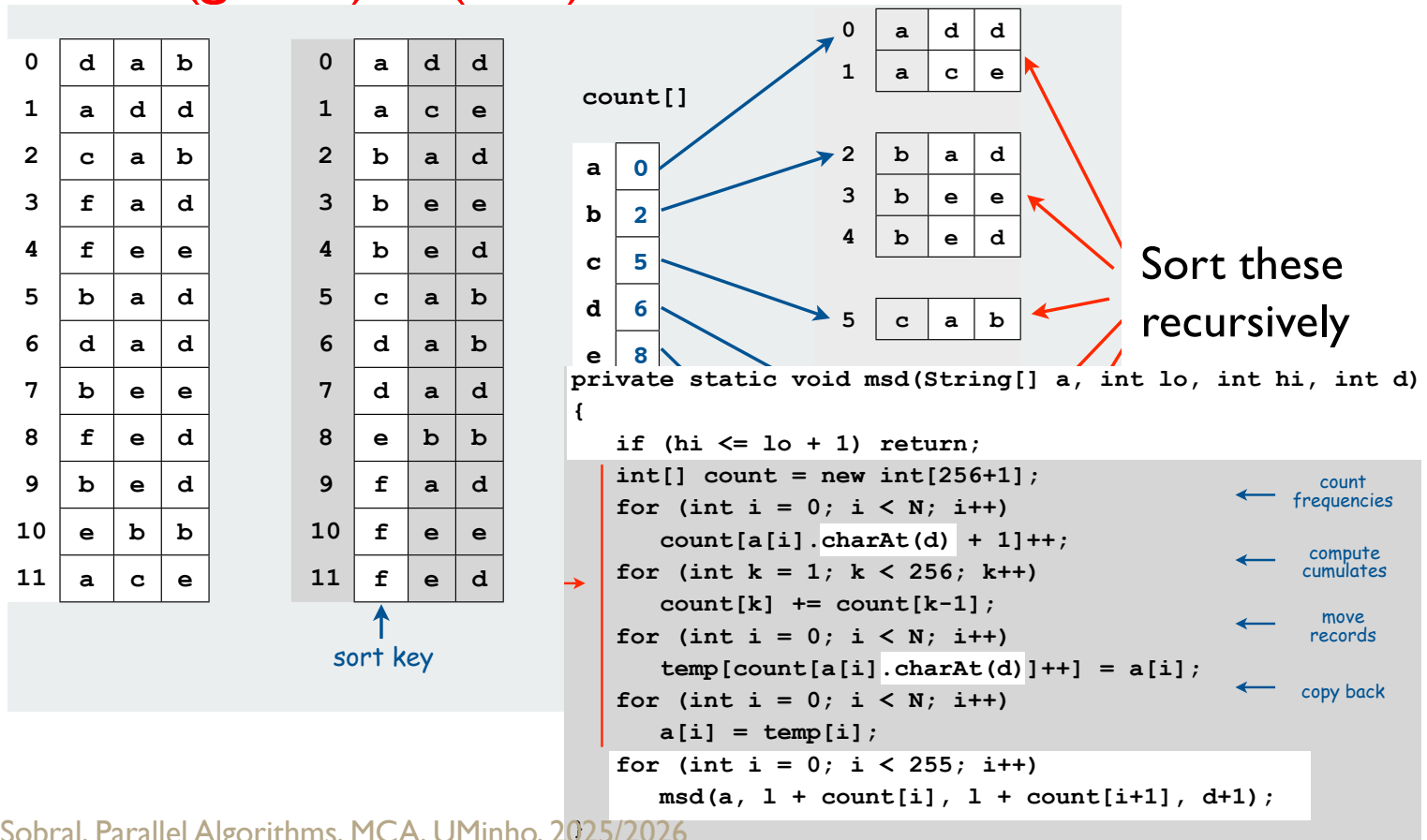
Parallel Sorting

- Locality: Radix LSD (e.g. string with 3 digits)
 - D steps through the full data: **$D \times N$ data moves**



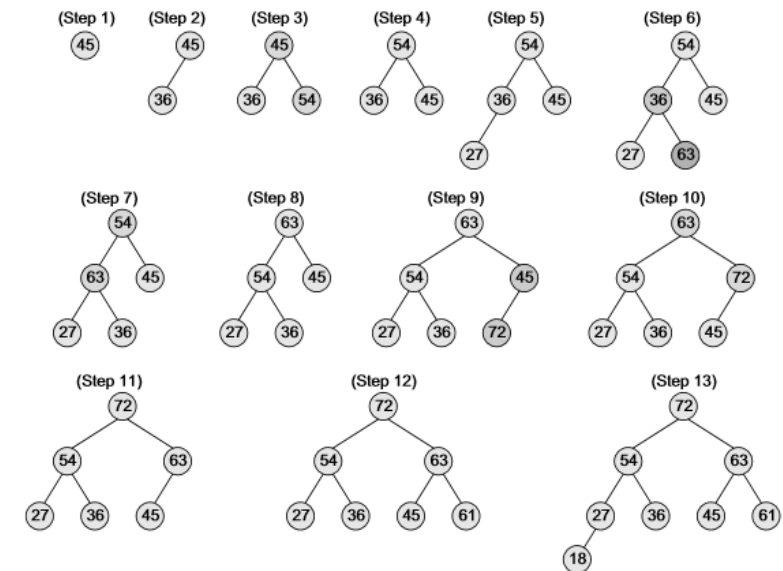
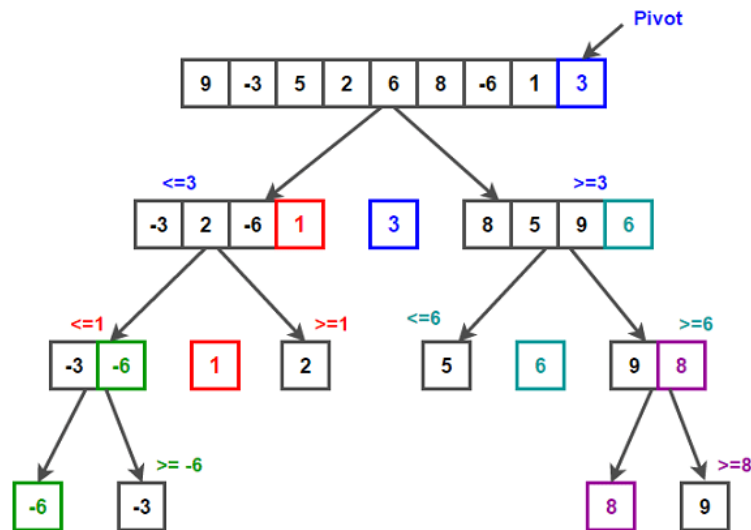
Parallel Sorting

- Locality: Radix MSD (e.g. string with 3 digits)
 - Partition data into K-sets: one step through full data:
 N (global) + $(D-1) \times N$ local data moves



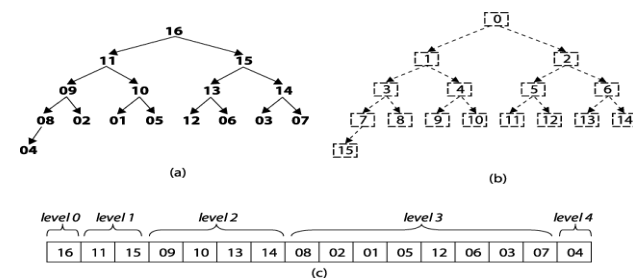
Parallel Sorting

- Locality: quick-sort (left) vs heap-sort (right)
 - Regular VS **irregular** data accesses



Note: heap-sort uses a **regular data structure**

Key insight: Both perform $n \times \log n$ steps through the data but heapsort shows more irregular data accesses



Parallel Sorting

- Locality of reference in sorting algorithms

Method	Locality of reference	Improvements
Quicksort	Good spatial locality + bad temporal locality on initial stages	Initial set partitioning using k keys
Mergesort	Good spatial locality + bad temporal locality on final merge stages	Single merge when data exceeds cache size
Heap sort	Bad	Cache aware trees + d-fan-out
Insertion sort	Bad	
Radix sort	Good when MSD first (only when processing LSDs)	Reduce the number passes through data

Parallel Sorting

- Parallelism in sorting algorithms

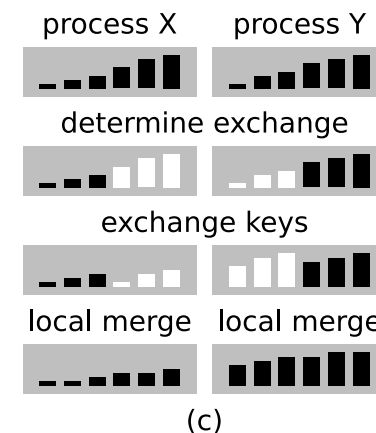
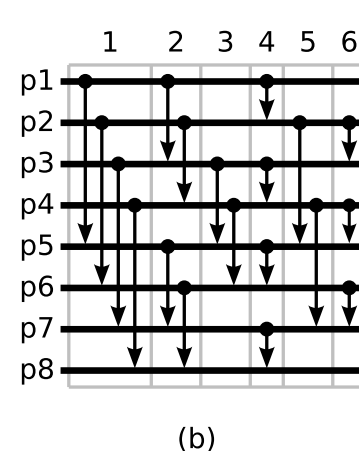
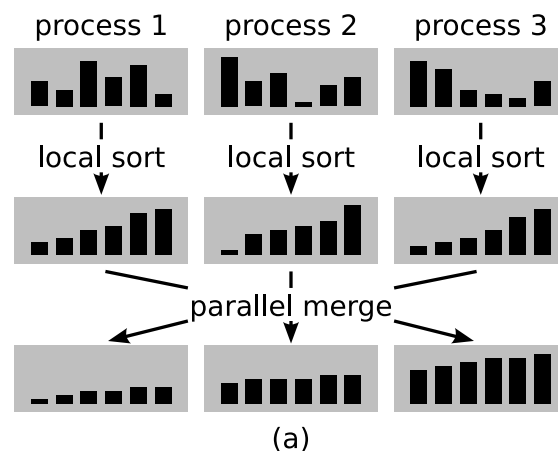
Method	
Quicksort	Sort sub-lists in parallel / start with p lists
Merge-sort	Merge p lists in parallel
Heap sort	???
Insertion sort	???
Radix sort	Sort sets of digits in parallel

Parallel Sorting (on distributed memory)

- Design issues:
 - Keys are initially distributed over processors
 - Intermediate stage for other parallel algorithms
 - Data properties
 - Partially-sorted data? (not in the scope of this lecture)
 - Exploitable parallelism
 - Merger-based
 - Splitter-based
 - Efficiency considerations for parallelism
 - Data movements [across processors]
 - Load balancing
 - Avoid idle time (e.g. sequential phases)

Parallel Sorting

- Parallel Merge-Sort
 - Locally sort each set
 - Exchange sets among processors



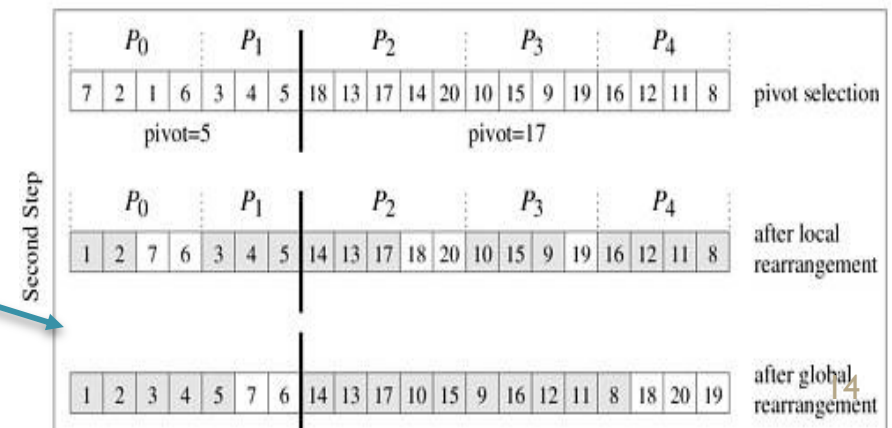
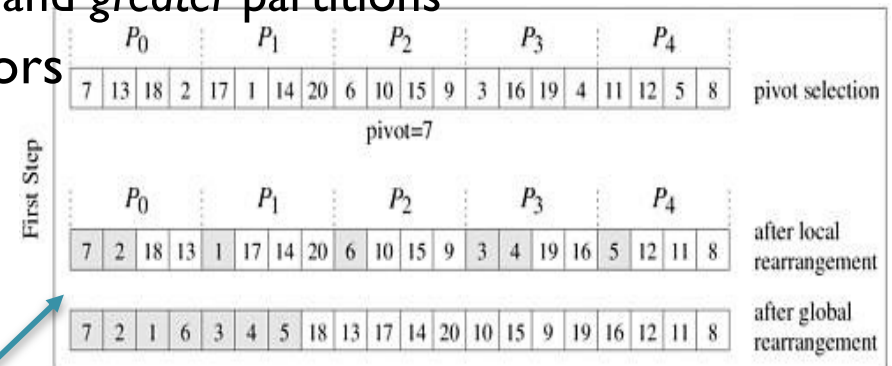
- Only effective when $n/p \sim I$
- Extensive data movements: when $n/p \gg I$

Parallel Sorting

- Parallel quicksort (simplified)

note: this used to illustrate the adaptation of a sequential algorithm, which is not the best path for sorting

- Master selects and broadcasts *pivot* key
- Each process locally splits using the *pivot*
 - Each processes holds *smaller* and *greater* partitions
- Divide [each set of] processors into two *subs-sets*
 - Send data to one processor on the other set
- Repeat the processes until $\#sets = \#p$
- Locally sort on each process p
- Complexity:
 - Requires $\log p$ communication steps



Parallel Sorting

- Parallel radix sort
 - Each processor is responsible by a subset of digit values
 - Sort and count the number of digit values
 - All-reduce the total number of digits
 - Send keys to the processor responsible for each digit range
 - Repeat for the next digit
 - Complexity:
 - LSD – **#D (number of digits)** communication steps
 - MSD – **One** communication step

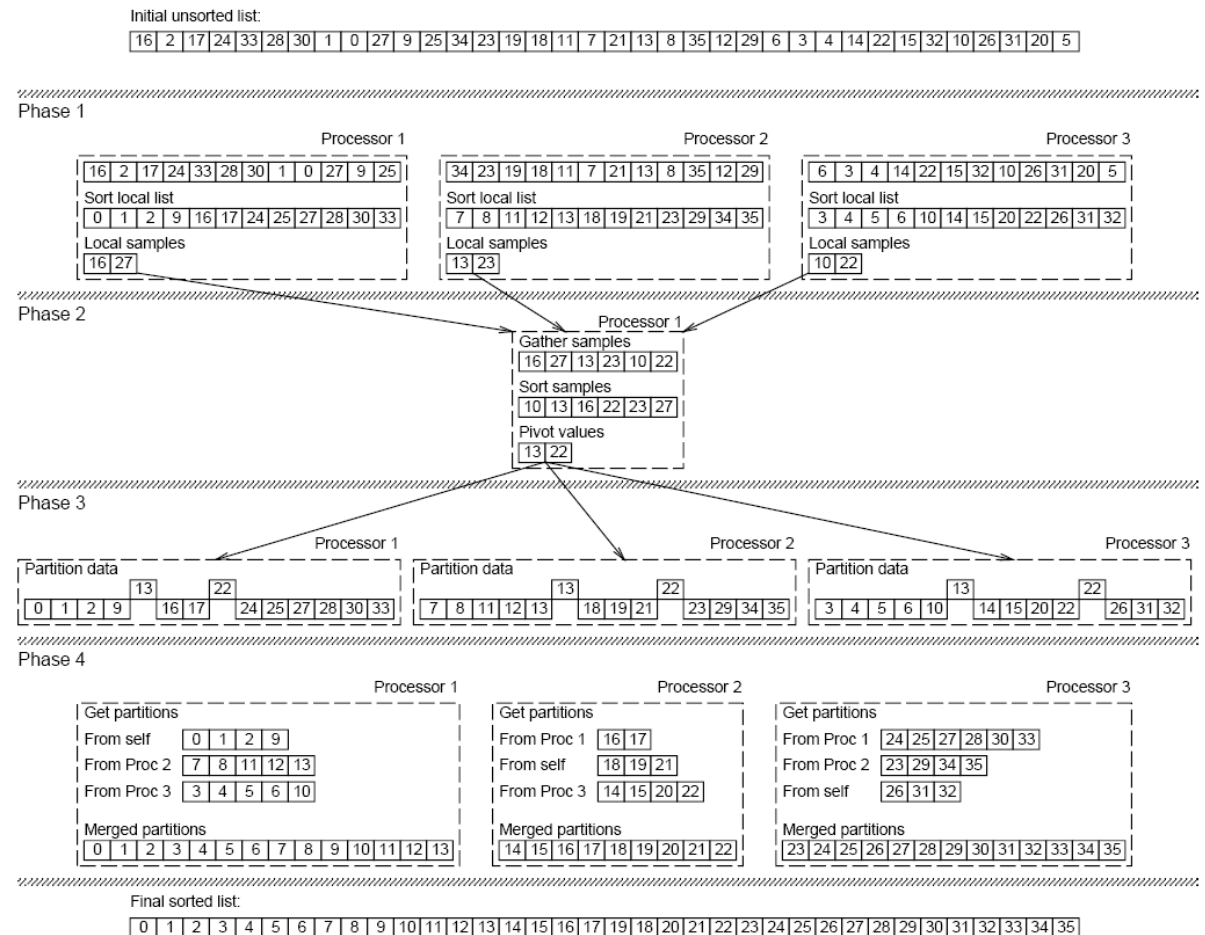
Parallel Sorting

- Parallelism in sorting algorithms
 - Sampling based
 - Split data into P sets using $p-1$ splitters
 - Each processor acts upon a local set
 - Minimizes data movements
 - Sampling alternatives
 - Regular sampling ($p^*(p-1)$ keys)
 - Not effective for large p
 - Random sampling
 - Histogram sampling

Parallel Sorting by Regular Sampling

(note: this is one of the best-known parallel sorting algorithms)

1. Divide the set into p disjoint sets and locally sort each set
 - Applies a local QuickSort
 - Selects $p-1$ local samples that uniformly divide each set into p subsets
2. Sort the $p*(p-1)$ samples and select best $p-1$ pivot keys
3. Partition each set using the $p-1$ pivot keys
4. Merge $p*p$ sets
 - Processor i merges the i partition



PL lecture

Parallelize **one** of given sort codes

- Shared Memory: OpenMP & gcc
- Input of size of 100 000 000 integer keys (32 bits)

Suggestion

1. Profile execution of the five sort implementations

```
perf stat -e instructions,cache-misses ./sort 1 1 100000000
```

2. Edit the file `sort.c` to identify the code of the best implementation. What parts of the algorithm can be executed in parallel?

Execute the following commands to collect a profile of the run-time of each part of the algorithm:

```
perf record ./sort <<variant>> 1 100000000  
perf annotate -l --stdio
```

3. Develop a parallel version of the sort with OpenMP? What is the maximum expected speed-up?

Provided sort implementations (in `sort.zip`)

```
1. void quicksort(int vetor[], int begin, int end)  
...  
2. void radixsortlsd(int vetor[], int size)  
...  
3. void heapsort(int vetor[], int n)  
...  
4. void mergesort(int vetor[], int begin, int end){  
...  
5. void radixsortmsd(int vetor[], int size)  
...
```